

Interactions Authoring Guide

Use this guide when you need to describe **screens, views, user actions, system responses, and navigation between screens**.

This guide is about **what the user sees, what the user can do, and how the interface moves from one state or screen to another**.

What belongs here

Capture things like:

- screens and views
- user actions on those screens
- feedback, validation, and response states
- transitions from one screen to another
- important links to stories, concepts, or rules

What does not belong here

- the whole business process — capture that in the Story guide
- full definitions of concepts — capture those in the Concepts guide
- technical implementation details of frontend components unless they change meaning

Core things to capture

Describe:

- the **screen name** and purpose
- what information the user **sees**
- what the user can **do**
- what the system **responds with**
- what happens after the action
- what errors, validations, or blocked states exist

Core relationships to capture

Make these links explicit in your text:

- screen -> action
- action -> result/state
- screen -> next screen
- screen -> concepts displayed or edited
- interaction -> story or user goal served
- interaction -> rule, decision, or test concern if important

Modus operandi

Walk through the interaction as if you were following a user:

1. identify the entry screen
2. describe what the user sees first
3. list available actions
4. describe the response to each important action
5. describe where the user goes next
6. capture error, validation, and empty states

Focus on user intent and system feedback, not on pixel-perfect layout.

Prompt set

Use questions like these:

- what does the user see first?
- what can the user do from here?
- what information is required before continuing?
- what happens after the user acts?
- what confirmations, errors, or warnings appear?
- where does the user go next?
- which concepts are being viewed or changed?

Free-text intake template

Use this template when gathering notes:

- **Screen name:**
- **Purpose:**

- **User type / actor:**
- **What the user sees:**
- **Main actions available:**
- **System responses:**
- **Validation or error states:**
- **Navigation:**
 - from:
 - to:
 - when:
- **Concepts shown or edited:**
- **Related story or goal:**

Worked example

Example: leave request form

- **Screen name:** Leave Request Form
- **Purpose:** let an employee submit a time-off request
- **What the user sees:** date fields, leave type, reason field, remaining balance summary
- **Main actions:** submit request, save draft, cancel
- **System responses:**
 - on valid submit, show confirmation and send request for review
 - on invalid dates, show a validation error
 - on policy conflict, show warning and block submission
- **Navigation:**
 - from employee dashboard to leave request form
 - from leave request form to confirmation page after successful submit
- **Concepts shown or edited:** employee, leave request, leave balance
- **Related story:** employee submits a leave request

How this becomes YAML

A technical modeler or AI agent can transform this into an interactions layer by:

- turning screens into named UI views
- turning actions into user-triggered interaction points
- turning navigation into directed transitions
- linking screens and actions to related stories, concepts, goals, or rules

You do not need to describe screen IDs or field-level schema structure.

Common mistakes

- describing layout but not user purpose
- naming actions without describing their results
- forgetting validation/error states
- forgetting navigation after success or failure
- mixing business-process detail and UI detail into one undifferentiated list
- omitting the concepts that the screen displays or changes